从 Newton-Schulz 到 Polar Express

一步步看懂 Muon 里的多项式迭代

目录

1 Modded NanoGPT：Muon 是怎么火起来的 **2**

1.1 Speedrun 是什么玩法 2

1.2 Muon 在 modded-nanogpt 里的角色 2

1.3 Polar Express 进 modded-nanogpt 3

1.4 对你的实践意义 3

2 背景：Muon 里那个 polar(M) 是啥 **4**

3 Newton-Schulz：经典做法 **4**

3.1 两种 NS 多项式的形态对比 5

3.2 Degree-3 NS 复合 1 – 5 次 5

3.3 Degree-5 NS 复合 1 – 5 次 6

3.4 两个版本的”台阶”对比 6

3.5 NS 的本质问题：小 x 处放大倍数有上限 7

4 Jordan 的野路子：为速度牺牲收敛性 **7**

5 Polar Express 的核心想法：每步换多项式 **8**

5.1 把问题写成数学优化 8

5.2 贪心策略 8

6 每步的最优多项式：等振荡 **8**

6.1 为啥必然”振荡” 8

6.2 对称性带来奇迹（Lemma C.2） 9

6.3 为啥贪心 = 最优? 9

7 具体算一遍：$\ell = 0.1, u = 1, T = 3$ **10**

7.1 Polar Express 三个多项式的画像 10

7.2 区间是怎么收缩的 11

7.3 Polar Express vs NS：从 $\ell_0 = 0.1$ 出发 11

8 总结 **11**

1 Modded NanoGPT：Muon 是怎么火起来的

讲完算法，回到背景。Muon 这套东西能进入大家视野，很大程度是因为一个叫做 **modded-nanogpt**[1] 的项目——直译”魔改 nanoGPT”。这个 repo 本身是个比赛性质的开源工程，最初的目标只有一个：

> *在单台 8×H100 节点上，把 GPT-2 训练到验证 loss 3.28，速度多快？*

这个目标叫做 **NanoGPT speedrun**（速通）。Karpathy 的原版 nanoGPT 用 AdamW 训 GPT-2 大概要 45 分钟到验证 loss 3.28；modded-nanogpt 通过一系列优化把这个时间一路压到了几分钟。Muon 就是这个过程中的核心改动之一。

1.1 Speedrun 是什么玩法

把训练当成游戏速通——和图 1.1 类似的赛道思维：固定终点（val loss ≤ 3.28），优化耗时，所有人在同一硬件配置（8×H100）上比谁更快。

NanoGPT Speedrun 进化史（示意）

为啥这件事重要？速通带来的好处不只是”训得快”：

- **研究效率**：训练 cycle 越短，研究者能跑的 ablation 越多，迭代越快。
- **基础设施基线**：speedrun 是个公开 leaderboard，所有人都能比较自己的优化器、kernel、架构改动到底有没有真实收益。
- **方法论筛选**：能在 speedrun 上活下来的技巧基本都是真有用的——毕竟谁都骗不了 wall clock。

Muon 就是这样从 speedrun 里”杀”出来的——Jordan 设计 Muon 时就是为了把 speedrun 时间再压一刀。结果不仅压成了，还发现 Muon 在更大模型上也 work，于是有了后来 Kimi K2 这种规模上的应用。

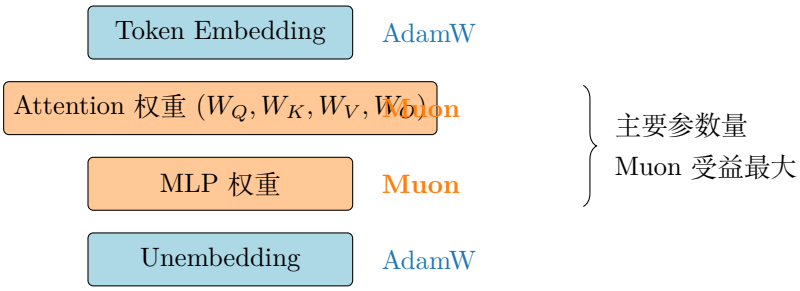
1.2 Muon 在 modded-nanogpt 里的角色

Modded-nanogpt 的训练规则是这样的：

- **Embeddings, unembeddings, 位置编码**用 AdamW 训。
- **所有其他 ≥ 2 维参数**用 Muon 训。

[1] https://github.com/KellerJordan/modded-nanogpt

为啥这么分？因为 Muon 的”把所有奇异值映射到 1”这个操作，对 embedding 这种本来就是 lookup 表的东西没意义——embedding 的列向量长度有信息含量，不能强行抹掉。但对 attention 和 MLP 的权重矩阵，Muon 的几何（spectral steepest descent）天然合适。



1.3 Polar Express 进 modded-nanogpt

文章作者把 Polar Express 提交到 modded-nanogpt 后被合并进了官方 repo——这是一个非常典型的”理论-实践”完整闭环：

1. Jordan 在 speedrun 里靠 grid search 找了个非收敛但前期快的多项式（Section 3）。
2. Polar Express 发现”前期快”和”收敛”其实可以同时拿到（Theorem 3.1），并且实现极其简单（30 行 PyTorch）。
3. 在 GPT-2 训练上验证了一致更优，于是被并入 speedrun 的官方实现。

1.4 对你的实践意义

如果你自己想用 Muon 训模型，目前最干净的路径是：

1. 直接 clone modded-nanogpt 看代码——Muon 优化器 + Polar Express 的实现都在里面。
2. 把它的 Muon 类抠出来用到自己模型上。注意三件事：
   - 区分 Muon 参数和 AdamW 参数（embedding 之类要排除）。
   - 学习率比 AdamW 大一个量级（典型 10^{-2} 量级）。
   - 用 5–6 步 Polar Express，再多没用（论文 Figure 5 的 ablation）。
3. 如果模型是 transformer，权重矩阵长宽比大于 4，可以考虑论文 Appendix J 的”分头并行”trick——不过那个收益要 embedding ≥ 1280 的大模型才看得出。

一句话总结：modded-nanogpt 是 Muon 的孵化器和验证场，Polar Express 是这个孵化器最近一次升级的核心算法贡献。两者绑在一起，让”用多项式逼近 polar 分解”这个老问题在 LLM 训练这个新场景里焕发了生命力。

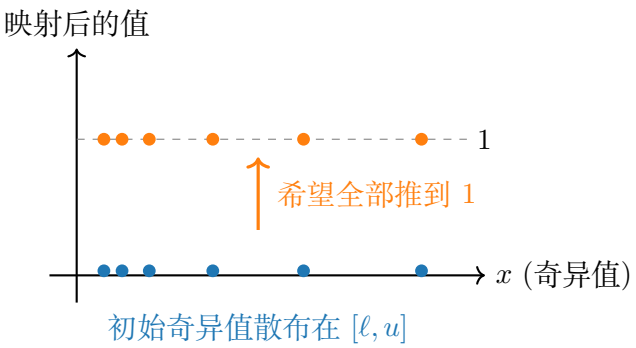
2 背景：Muon 里那个 polar(M) 是啥

Muon 优化器的更新规则是：

$$W_{t+1} = W_t - \lambda \cdot \mathrm{polar}(M_t)$$

其中 M_t 是动量，$\mathrm{polar}(M)$ 把矩阵的 SVD $M = U\Sigma V^\top$ 中的奇异值**全部映射到 1**：

$$\mathrm{polar}(M) = UV^\top$$



为啥不直接 SVD？ GPU 不喜欢——SVD 不是矩阵乘法、并行度差。我们要用**只含矩阵乘法**的方法：迭代多项式。

对一个 odd 多项式 $p(x) = ax + bx^3 + cx^5$，作用到矩阵上等价于：

$$p(M) = aM + bM(M^\top M) + cM(M^\top M)^2$$

全是 matmul，GPU 友好。每一次迭代相当于在每个奇异值上独立地作用一次 p。

3 Newton-Schulz：经典做法

最经典的是 Newton-Schulz 迭代。每一步用同一个 odd 多项式，反复作用：

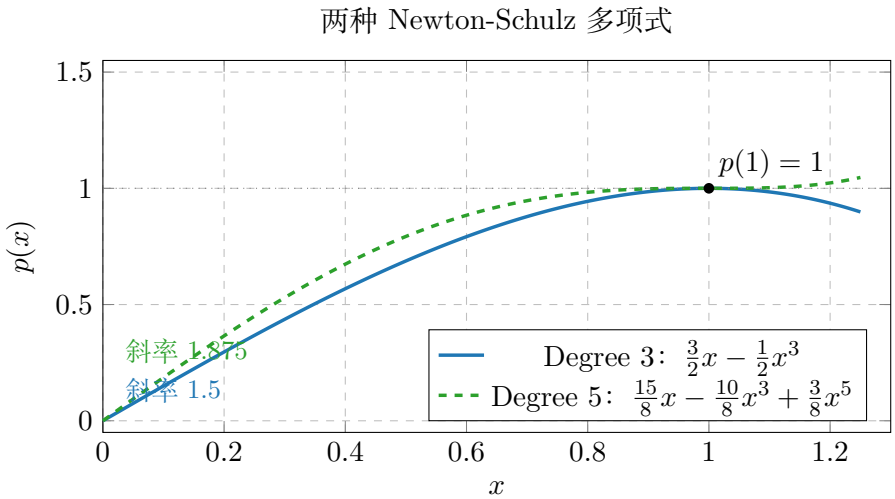
$$X_{t+1} = p_{NS}(X_t)$$

两个常用版本：

- **Degree 3**：$p_{NS}^{(3)}(x) = \frac{3}{2}x - \frac{1}{2}x^3$
- **Degree 5**：$p_{NS}^{(5)}(x) = \frac{15}{8}x - \frac{10}{8}x^3 + \frac{3}{8}x^5$

它们都满足 $p(1) = 1$ 且导数在 $x = 1$ 处为 0（degree 5 的甚至二阶导也为 0），所以 $x = 1$ 是**超稳定不动点**。

3.1 两种 NS 多项式的形态对比

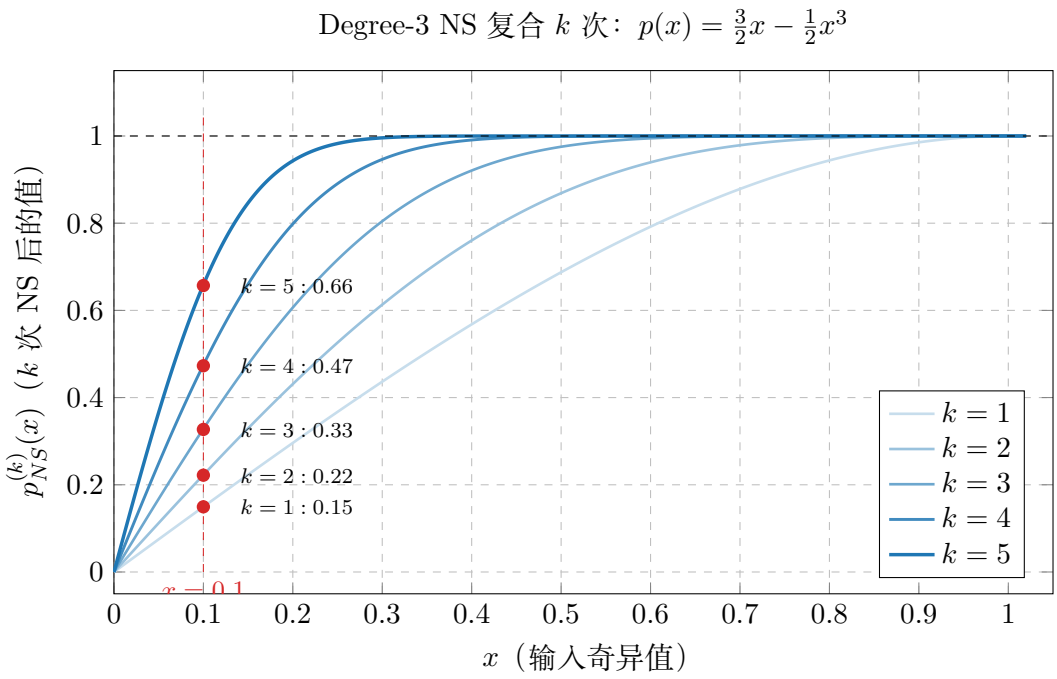


关键差别:

- Degree 5 在 $x \to 0$ 处的斜率更大（$1.875 > 1.5$），所以在小奇异值处放大得更快。
- Degree 5 的曲线" 贴 $y = 1$" 的台阶更宽——一次迭代能把更多 x 推到接近 1。
- 但 degree 5 每步多一次矩阵乘法（计算 $X(X^\top X)^2$）。

3.2 Degree-3 NS 复合 1 – 5 次

把 NS 自己跟自己复合：$p_{NS}^{(k)} = \underbrace{p_{NS} \circ \cdots \circ p_{NS}}_{k\ 次}$。



怎么读这张图：颜色由浅到深 = 复合次数 1 到 5。每条曲线告诉你"k 次 NS 后，输入 x 变成了多少"。

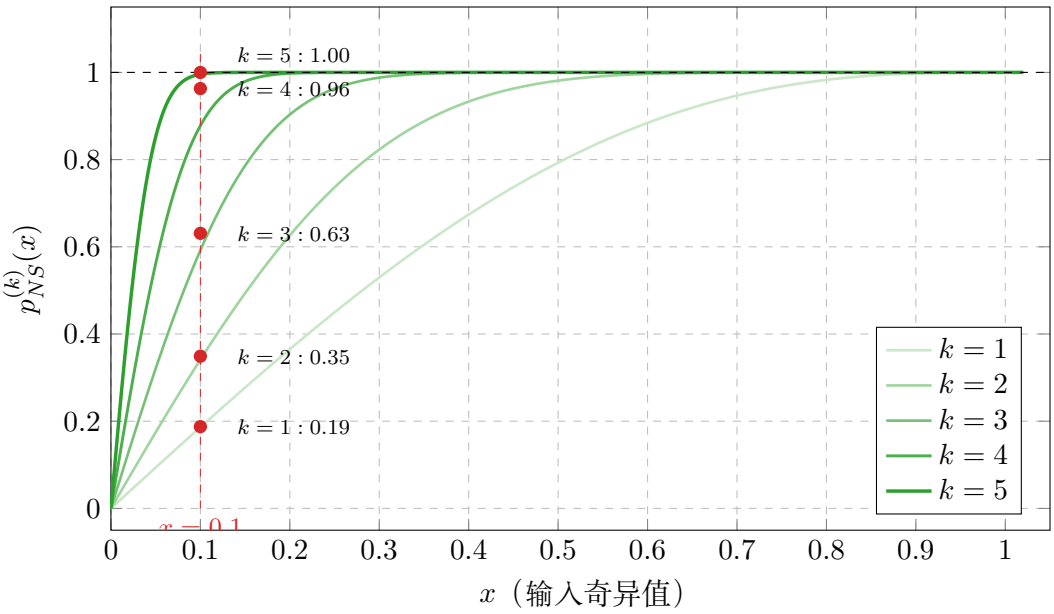
$x = 0.1$ **这一列追踪**（红点）：

$$0.1 \xrightarrow{\text{NS}} 0.15 \xrightarrow{\text{NS}} 0.22 \xrightarrow{\text{NS}} 0.33 \xrightarrow{\text{NS}} 0.47 \xrightarrow{\text{NS}} 0.66$$

跑了 5 次还没到 1！Degree 3 NS 在小 x 处推进非常缓慢。

3.3 Degree-5 NS 复合 1−5 次

Degree-5 NS 复合 k 次：$p(x) = \frac{15}{8}x - \frac{10}{8}x^3 + \frac{3}{8}x^5$



$x = 0.1$ **这一列追踪**：

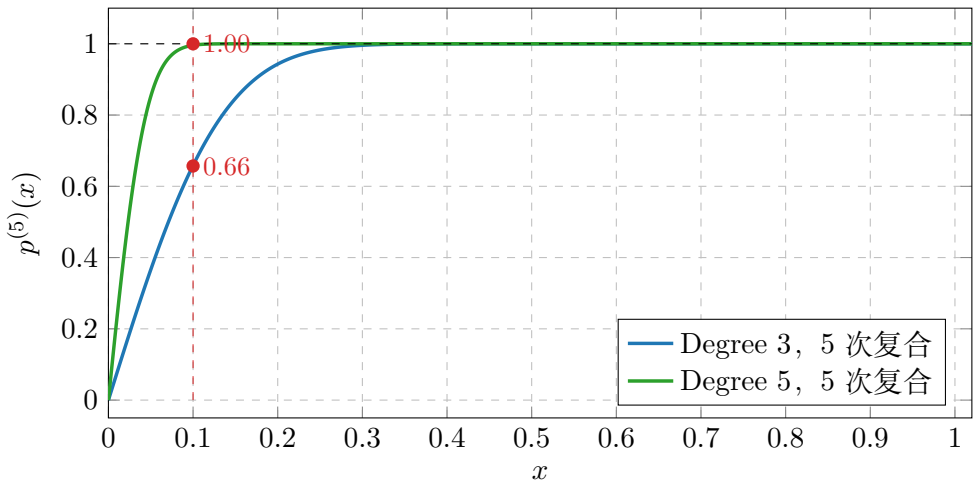
$$0.1 \xrightarrow{\text{NS}_5} 0.19 \xrightarrow{\text{NS}_5} 0.35 \xrightarrow{\text{NS}_5} 0.63 \xrightarrow{\text{NS}_5} 0.96 \xrightarrow{\text{NS}_5} 1.00$$

Degree 5 显著比 degree 3 快——5 步基本就到 1 了。这就是为啥实践中大家都用 degree 5。

3.4 两个版本的”台阶”对比

把两个 degree 复合 5 次后的曲线叠在一起，能看清差距：

5 次复合后：degree 3 vs degree 5



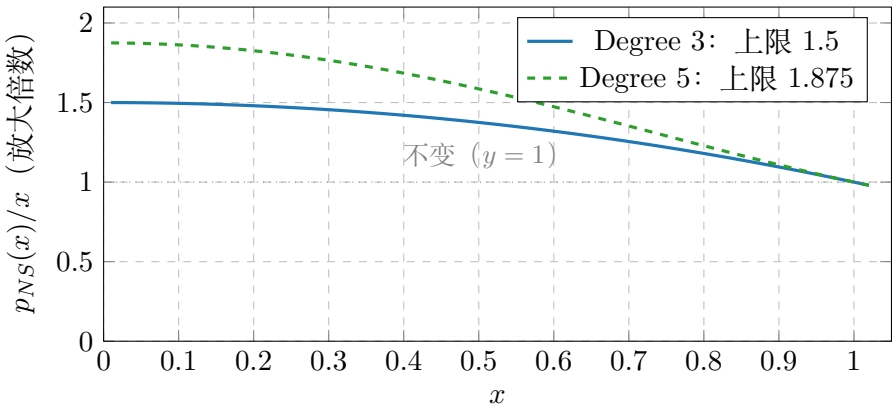
Degree 5 把”已收敛”的范围拓展到约 $x \geq 0.1$，而 degree 3 还在 $x = 0.1$ 处只爬到 0.66。

3.5 NS 的本质问题：小 x 处放大倍数有上限

为啥小 x 这么慢？看每个 NS 多项式在 $x \to 0$ 处能”放大几倍”：

$$\lim_{x\to 0} \frac{p_{NS}^{(3)}(x)}{x} = \frac{3}{2}, \qquad \lim_{x\to 0} \frac{p_{NS}^{(5)}(x)}{x} = \frac{15}{8} = 1.875$$

NS 在每个 x 上的”放大倍数”



NS 死板的根源：无论 degree 多少，$x \to 0$ 处的放大倍数是**固定常数**。如果你的 $\sigma_{\min} = 10^{-3}$，degree-5 NS 第一步能把它推到 $\approx 1.875 \times 10^{-3}$——还是很小。

Polar Express 的核心想法：何必死守这个常数？**第一步就用更大的放大倍数！**

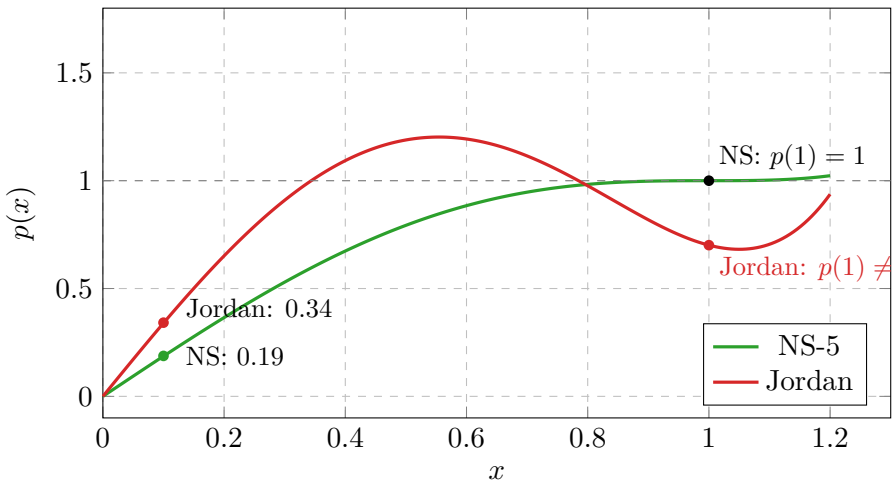
4 Jordan 的野路子：为速度牺牲收敛性

Muon 的开发者 Jordan 想：训 LLM 不需要把 polar 算特别准，能不能找个多项式**前期更猛**？他在 GPU 上 grid search 调出：

$$p_J(x) = 3.4445x - 4.7750x^3 + 2.0315x^5$$

小 x 处放大 3.4445 倍，比 NS 的 1.875 大近一倍。

Jordan vs Newton-Schulz（degree 5）



致命代价：Jordan 多项式不满足 $p(1) = 1$！它在 $x = 1$ 附近震荡，没有不动点。

后果：迭代会卡在某个误差 ≈ 0.3 的水平不再下降——**Jordan 方法根本不收敛**，只是" 前几步快"。You 的方法（用 6 个手调多项式轮流用）类似，不收敛但更快。

这就引出 Polar Express 的核心问题：**能不能既前期快，又能收敛？**

5 Polar Express 的核心想法：每步换多项式

NS 的问题在于**每步都用同一个多项式**。如果**每一步换一个多项式**，针对当前奇异值所在的区间动态调整呢?

5.1 把问题写成数学优化

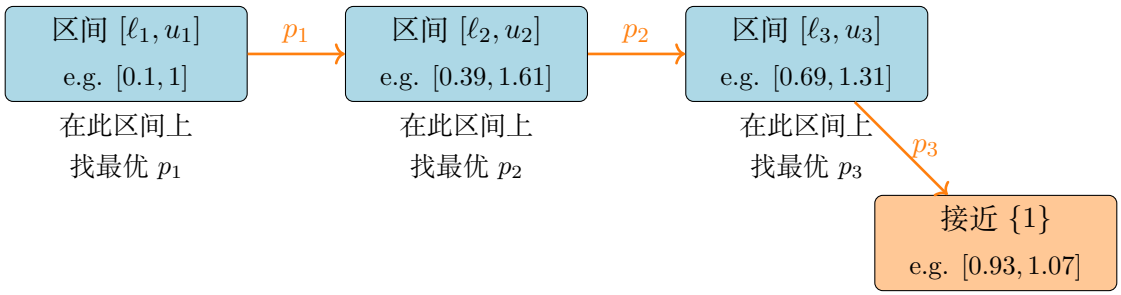
设奇异值在 $[\ell, u]$ 内。要找 T 个 odd 多项式 $p_1, \ldots, p_T$，使得复合 $p = p_T \circ \cdots \circ p_1$ 满足

$$\boxed{\min_{p_1,\ldots,p_T} \max_{x\in[\ell,u]} |1 - p(x)|}$$

即：复合多项式在整个区间上最接近常数 1（worst-case 误差最小）。

5.2 贪心策略

贪心：每一步只看当前区间，挑当前最优的单步多项式。



每一步的任务：

$$p_t = \arg\min_{p\in\mathbb{P}_d^{\text{odd}}} \max_{x\in[\ell_t,u_t]} |1 - p(x)|$$

然后新区间 $[\ell_{t+1}, u_{t+1}] = [p_t(\ell_t), \max_{[\ell_t,u_t]} p_t]$。

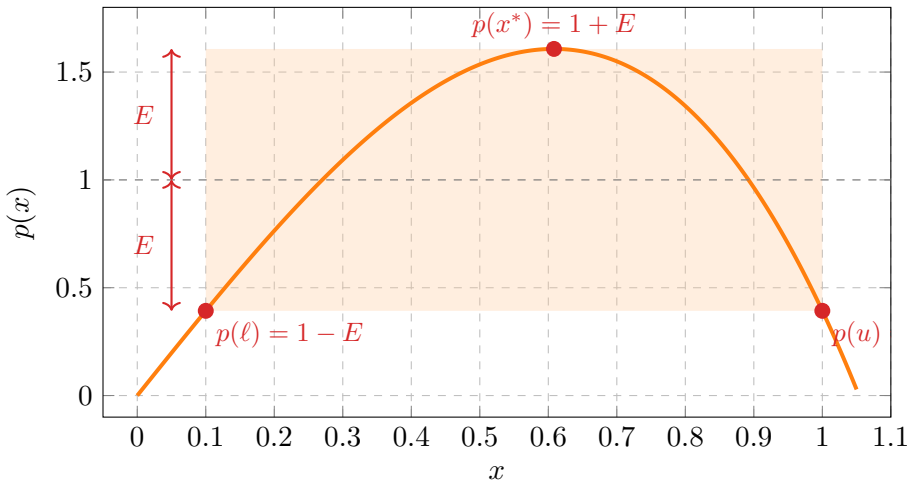
核心定理（文章 Theorem 3.1）：贪心 = 全局最优。

6 每步的最优多项式：等振荡

6.1 为啥必然" 振荡"

在区间 $[\ell, u]$ 上找 odd cubic $p(x) = ax + bx^3$ 逼近常数 1，最优解**必然在 3 个点上交替达到 ± 最大误差**。这叫 equioscillation。

最优 odd cubic 在 $[0.1, 1]$ 上逼近 1（等振荡）



物理直觉：如果误差不是这样”摸着上下界”，你总能稍微调一下让最大误差变小——也就是说还没到极限。最优解必然把误差压到极限。

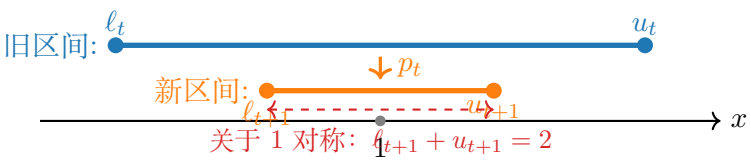
6.2 对称性带来奇迹（Lemma C.2）

由等振荡 + odd 函数的性质：

- ℓ 必是 $p^\star$ 的最小值点
- $\max p^\star = 2 - p^\star(\ell)$（关于 1 对称）
- 最大误差 $E = 1 - p^\star(\ell)$

也就是说，应用 $p^\star$ 后**新区间总是关于 1 对称**：

$$[\ell_{t+1}, u_{t+1}] = [p^\star(\ell_t),\ 2 - p^\star(\ell_t)]$$



6.3 为啥贪心 = 最优？

因为这种对称性，”让当前误差最小”和”给未来留下最好状态”是**同一件事**：

$$\begin{aligned} & \text{最小化当前误差} 1 - \ell_{t+1} \\ \iff\ & \text{最大化} \ell_{t+1} \\ \iff\ & \text{最小化} u_{t+1} \\ \iff\ & \text{最小化区间宽度} u_{t+1} - \ell_{t+1} \end{aligned}$$

所以贪心**不存在”短视”问题**。

7 具体算一遍：$\ell = 0.1, u = 1, T = 3$

用 degree-3 闭式解（文章公式 17）：

$$p(x) = \beta \cdot p_{NS}^{(3)}(\alpha x), \quad \alpha = \sqrt{\frac{3}{u^2+\ell u+\ell^2}}, \quad \beta = \frac{4}{2+\ell u(\ell+u)\alpha^3}$$

- **第 1 步**：$[0.1, 1]$，$\alpha_1 \approx 1.644$，$\beta_1 \approx 1.607$，

$$p_1(x) \approx 3.964x - 3.571x^3$$

  应用后：$\ell_2 = 0.393, u_2 = 1.607$。

- **第 2 步**：$[0.393, 1.607]$，

$$p_2(x) \approx 1.850x - 0.549x^3$$

  应用后：$\ell_3 = 0.693, u_3 = 1.307$。

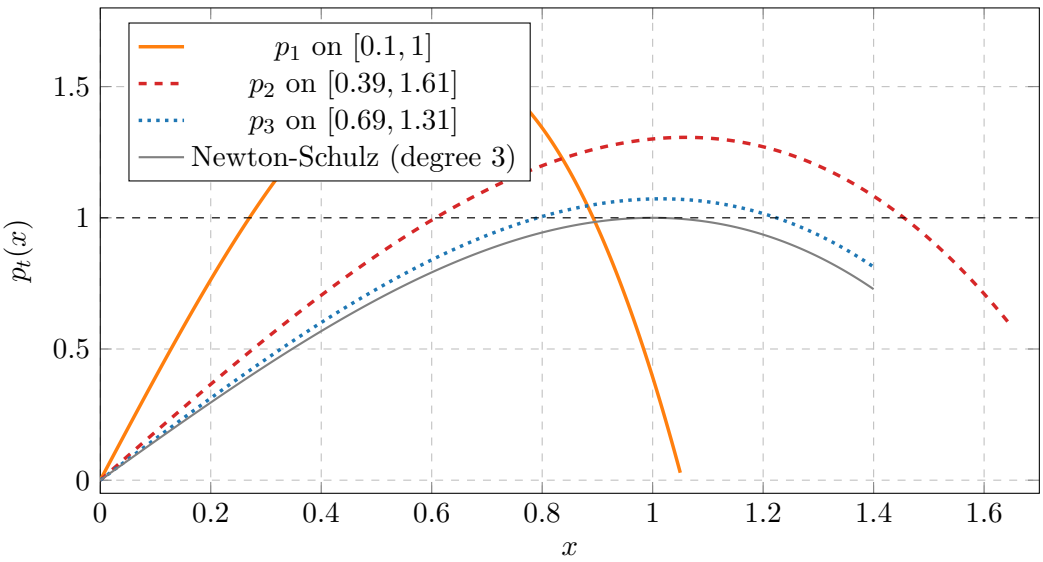
- **第 3 步**：$[0.693, 1.307]$，

$$p_3(x) \approx 1.584x - 0.512x^3$$

  应用后：$\ell_4 = 0.928, u_4 = 1.072$。

7.1 Polar Express 三个多项式的画像

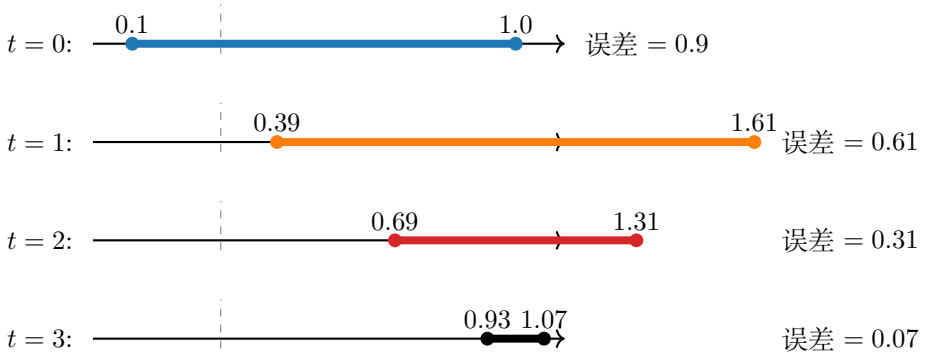
Polar Express 的三个多项式（不同区间上的最优）



观察：

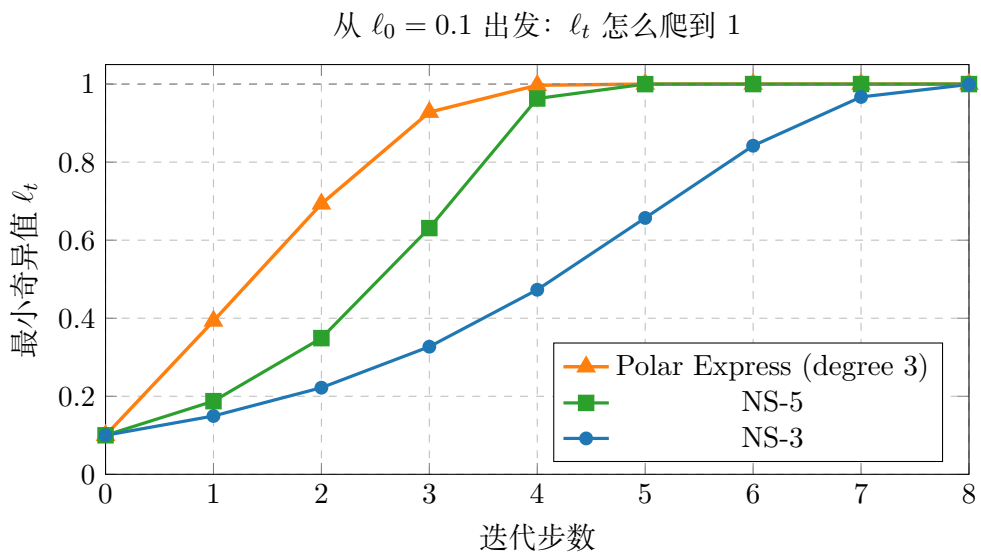
- p_1（橙色）系数最大、最”激进”——它要把宽广的 $[0.1, 1]$ 一次推到 1 附近。看小 x 处的斜率：$p_1'(0) \approx 3.96$，比 NS 的 1.5 大 2.6 倍！
- p_3（蓝色虚线）几乎就是 NS——区间已经接近 $[1, 1]$，不需要那么激进。
- Polar Express 的精髓：**动态调整放大倍数**，初期用大倍数，后期用小倍数。

7.2 区间是怎么收缩的



每一步区间都**关于 1 对称地收缩**（这就是 Lemma C.2 的对称性）。

7.3 Polar Express vs NS：从 $\ell_0 = 0.1$ 出发



观察：

- PE 第 1 步把 0.1 推到 0.39，比 NS-5 的 0.19 快了一倍多。
- PE 4 步基本收敛；NS-5 需要 5 步；NS-3 需要 8 步。
- PE-3（degree 3）能跟 NS-5（degree 5）比，意味着用更低 degree 也能拿到 NS-5 的速度——节省每步的计算量。

8 总结

方法	每步多项式	收敛性	前期速度
NS (degree 3)	固定（$\frac{3}{2}, -\frac{1}{2}$）	收敛到机器精度	慢
NS (degree 5)	固定（$\frac{15}{8}, -\frac{10}{8}, \frac{3}{8}$）	收敛到机器精度	中等
Jordan	固定（grid search）	**不收敛**（卡 ≈ 0.3）	快
You	6 个 hand-tuned	**不收敛**	略快于 Jordan
Polar Express	**每步动态最优**	收敛	**快**

Polar Express 三个核心 idea：

1. **每步换多项式**（不像 NS 死板）。
2. **每步的多项式由 minimax 优化得出**（不像 Jordan/You 是 grid search 蒙的）。
3. **贪心 = 全局最优**（依赖等振荡定理 + odd 多项式空间的对称性）。

加上 bfloat16 数值稳定 trick（safety factor、cushioning），就成了一个 30 行 PyTorch、可以直接替换 Muon 里 polar 计算的 drop-in module。